

操作系统研讨课

Course: B0911011Y

卢天越

2026 春季学期



Project 4 Virtual Memory

2025.4.24



今日课程内容

- Project 4介绍
- Project 3验收
- 答疑



截止P3

- 通过shell进行用户-系统交互
- 具备调度多个进程的能力
- **问题：**一个进程强行访问另外一个进程、甚至内核的物理地址，不会报错
- **需要虚拟内存，保证安全性**



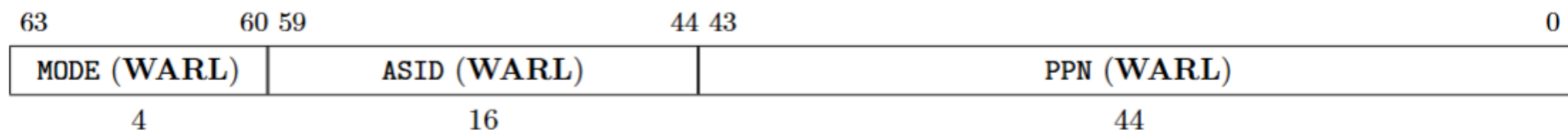
虚拟内存

- 首先需要打开虚拟内存机制
 - 在P4之前，虚拟内存机制没有打开，CPU认为地址是物理地址
- 打开后，CPU将所有指令中的地址认为是虚拟地址，通过TLB和页表进行地址翻译
 - TLB是硬件结构，对软件透明
 - 页表由软件写入，格式是硬件规定好的，供硬件自动查询

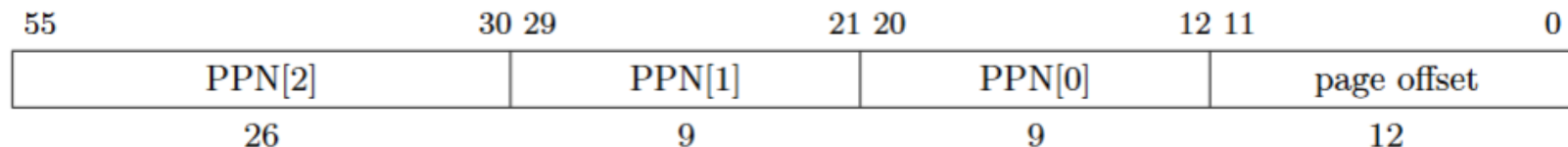


虚存相关CSR

- satp寄存器：硬件根据其中的物理地址找到页表的位置



- 多级页表：
 - 从根节点可以找到多级页表位置
 - Sv39: 39位虚拟地址，硬件自动分割地址



SV39页表项

- 每一级页表的页表项格式相同

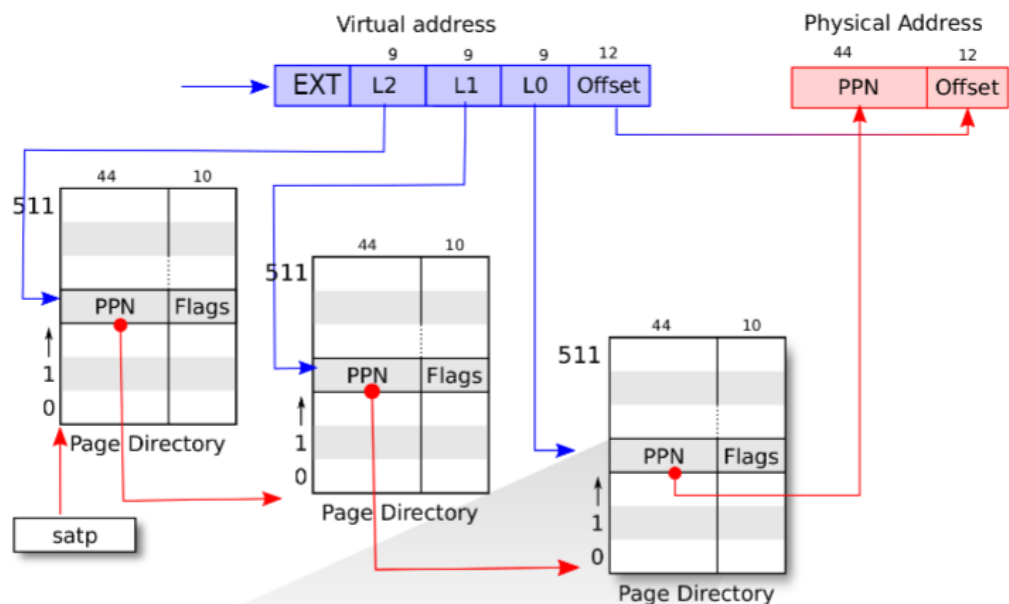
63	54 53	28 27	19 18	10 9	8	7	6	5	4	3	2	1	0
<i>Reserved</i>	PPN[2]	PPN[1]	PPN[0]	RSW	D	A	G	U	X	W	R	V	
10	26	9	9	2	1	1	1	1	1	1	1	1	

- 重要的标志位：
 - R & W & X: 权限
 - V: Valid, 是否有效
 - RWX都是0, 但是V为1, 代表什么?
 - 有效, 但是不允许所有权限? 毫无意义
 - 所以**用来代表非叶子节点, PPN指向下一级页表**



地址翻译过程

- 硬件自动完成，OS需要根据这个过程写好页表
 - 任何一步出错都会触发缺页异常（page fault）
 - 有时需要通过查页表确定具体原因



Page table walk

- 硬件自动进行地址翻译的过程（忽略TLB）：
 - 根据satp寄存器找到第一级页表
 - 在每一级页表中，根据RWXI四个标志位判断是否是叶子节点
 - 如果不是叶子节点，根据内容查找下一级
 - 如果是叶子节点，得到物理地址
 - 如果三级页表，VPN有27位，offset有12位，每个页面4KB大小
 - 如果两级，VPN有18位，offset有21位，每个页面2MB大小，S-core可以只实现两级页表



对TLB需要做什么

- TLB由硬件自动填入和换出，因此大部分情况软件不需要处理TLB
- 例外情况：由于更改页表的时候，有可能导致页表内容与TLB不一致，因此需要**手动对TLB内容进行清空**
 - 使用sfence.vma指令
 - 为了简单，可以每次修改页表时都执行sfence.vma指令，不需严格区分当前TLB内容是否一致



任务1：启动虚存机制进入内核

- 开启虚存机制之后，CPU将所有地址视为虚地址，所以在此之前就要准备好页表
 - 首先，需要一段在物理地址上运行的代码，由它来准备页表
 - Bootblock.S, start.S, boot.c
 - 之后切换成使用内核虚地址
 - head.S
 - 内核虚地址是物理地址前面加上0xffffffffc0的前缀，方便从物理地址到内核虚地址的转换
 - pa2kva、kva2pa函数



任务1续：虚存启动用户程序

- 为用户程序建立页表并把用户程序拷贝到指定分配好的物理页面
 - A-core的同学请将Makefile里面的用户入口地址改为0x10000，也就是说，**所有用户程序使用相同的用户虚地址入口**
 - 在Makefile文件中找到USER_ENTRYPOINT
 - 必须使用三级页表，4KB大小的页面
 - 栈空间也要准备好页面
 - exec时传递参数需要进行参数拷贝，此时会存在跨用户空间的情况，需要相应修改



任务2：动态页表和按需调页

- 如果为用户程序一开始就分配好所有页面和页表，就不会发生缺页
 - 但是存在资源浪费，进程所需资源不必在初始化时就分配好，进程运行过程中也可能动态申请内存
- 一开始只有虚拟地址，程序启动后会发生缺页异常，在中断处理中分配物理页面，完成虚实地址的映射
 - 触发缺页异常的地址会存在stval寄存器中
 - 测试程序会随机访问三个虚拟地址，进行动态映射
- 从任务2开始，仅A-core同学需要完成



任务3：换页机制

- 换页（page swap）：在物理内存不足时，将部分虚拟页面对应的内容写入磁盘中，待到需要使用时再换回物理内存
- 如果虚拟页面对应的内容在磁盘中，会触发缺页异常，此时需根据实际情况，进行换页，而非分配新的物理内存
- 测试程序：需同学自行实现，在内核中限制物理内存使用大小，并通过访问顺序，展示页替换算法



任务3：换页机制

- 注意事项：
 - 写入磁盘需要使用sd_write接口，注意地址512B对齐，否则会报错卡死
 - 在QEMU上调试，需要使用padding增加镜像大小
 - QEMU认为image文件即为磁盘，大小也等于image文件大小

```
dd if=/dev/zero of=image oflag=append conv=notrunc bs=64MB count=1
```

- 上板不需要这个命令



任务4：多线程mailbox

- 在虚存之前，进程和线程概念并没有明确区分，现在可以明确了
 - 进程拥有独立的虚拟地址空间，而一个进程的多个线程共享虚拟地址空间
- 提供了多线程mailbox的测试程序 mailbox.c
 - 三个进程，每个进程各启动两个线程进行mail收发
 - 请大家根据测试程序内容，实现线程接口



任务5：共享内存

- 如果两个进程将各自的虚页映射到相同的物理页面上，那么该页面就成为两个进程的共享页面，可以更方便的进行数据通信
- 测试程序consensus.c，利用互斥锁和barrier来进行同步，访问一块共享内存上的数据
 - 需要实现共享内存的申请和回收
 - 注意回收时，需要看是否有其他进程还在使用
- 从任务5开始仅需C-core同学完成



任务6：内存保护

- 通过修改页表中的权限位，可以实现对某个页面进行类似只读/不可执行这样的权限管理，
 - 实现mprotect系统调用，对某一块内存地址进行权限管理
- 测试程序需同学自行完成，通过访问某个不具备特定缺陷的内存地址，触发缺页异常，并在中断中判断出错误原因是由页面保护导致的权限错误即可



Project 4任务总结

- S-core: 任务1
- A-core: 任务1-4
- C-core: 任务1-6



P4 Design review

- 介绍你的设计，覆盖以下问题：
 - 描述页表结构、页表项内容
 - 描述在虚存上加载用户程序的过程
 - 描述缺页异常的处理流程，注意结合你要完成的任务区分多种缺页原因
 - 页替换策略准备如何实现
 - 如何实现线程调度，涉及哪些数据结构和函数的修改



注意事项

- P4的功能与P5、6相对独立，因此今天将同时发布P4-6的任务书。同学们可以根据自己实际情况选择完成顺序
 - P3的同步原语、双核等功能也相对独立，在完成shell功能后就可以开始P4任务
 - P4的任务一最难，后续任务简单
- 考虑大家的进度，五一之后的课程安排准备进行调整，请大家根据自己的实际情况投票



五一后课程

- 方案一：多给P2-3时间
 - P2A-core延长到8号
 - P3A-core延长到15号
 - P4-6的C-core的ddl依然为15、29、最后一次
- 方案二：多给P4-6时间
 - P2、3不延长
 - P4-6的C-core的ddl延后为29、5、最后一次
- 最后一次课调到端午节那周
 - 看大家的选课情况

五月(May.)					六月(Jun.)		
9	10	11	12	13	14	15	16
27	4	11	18	25	1	8	15
28	5	12	19	26	2	9	16
29	6	13	20	27	3	10	17
30	7	14	21	28	4	11	18
1 劳动节	8	15	22	29	5	12	19 端午节
2	9	16	23	30	6	13	20
3	10	17	24	31	7	14	21



谢谢!

下节课: 5月8日, P4 design review
Any questions?

